

FACULTY SHARED-SERVER DEPLOYMENT GUIDE

Deployment situation	Shared Ubuntu machine (headless server and a GUI machine may both be used)
Target hardware	GMKtec EVO-X2 — AMD Ryzen AI Max+ 395 (“Strix Halo”), Radeon 8060S iGPU (RDNA 3.5, gfx1151), unified LPDDR5X memory
New architecture	Two isolated containers: Streamlit/RAG + llama.cpp/Gemma
Knowledge base	<code>dbp_khidmatnasihat_clean_atomic</code> — 33,320 items
Important change	Do NOT install LM Studio, Conda, Python, ROCm, or another Docker Engine on the host

This replaces the **NVIDIA/RTX 5090 edition of this guide**. That version assumed a discrete NVIDIA GPU (`nvidia-smi` , CUDA images, the NVIDIA Container Toolkit). None of that exists on this hardware. The GMKtec EVO-X2 has **no discrete GPU** — its Radeon 8060S is an integrated GPU that shares system RAM (unified memory), and the whole NVIDIA-specific toolchain is replaced with **Vulkan**, which is the simplest and best-performing backend for this chip.

STOP

Do not reinstall Docker. Do not edit `/etc/docker/daemon.json` . Do not install ROCm. Do not restart Docker unless faculty IT explicitly approves it — this shared machine already runs other containers.

What Changed From the NVIDIA Version (Quick Reference)

NVIDIA edition	AMD/GMKtec EVO-X2 edition
<code>nvidia-smi</code>	No such tool. Use <code>vulkaninfo</code> , <code>ls /dev/dri /dev/kfd</code>
NVIDIA Container Toolkit + <code>--gpus all</code>	Not needed. GPU access is <code>--device=/dev/dri --device=/dev/kfd</code> plus <code>video / render</code> group membership
<code>nvidia/cuda</code> test image	Any base image + <code>mesa-vulkan-drivers vulkan-tools</code>
llama.cpp CUDA server image	llama.cpp Vulkan server image (<code>ghcr.io/ggml-org/llama.cpp:server-vulkan</code>)
Logs show “CUDA/GPU offload”	Logs show “Vulkan devices found... RADV GFX1151”
Fixed 32GB VRAM	Unified memory — GPU draws from shared system RAM
<code>LLAMA_HF_REPO</code> (GGUF file)	Unchanged — GGUF models work on any backend

1. New Architecture

```
Ubuntu shared host (server or GUI machine)
|
+-- Existing faculty Docker applications  <-- DO NOT TOUCH
|
+-- dbp-chatbot Compose project
    |
    +-- chatbot container
    |   +-- Streamlit
    |   +-- Phase 4 retrieval/reranking
    |   +-- Phase 5 generation client
    |   +-- BGE-M3 + reranker (CPU — unchanged)
    |   +-- ChromaDB mounted read-only
    |
    +-- llm container
        +-- llama.cpp Vulkan server
        +-- Gemma 3 4B IT GGUF
        +-- Radeon 8060S iGPU (gfx1151) via /dev/dri + /dev/kfd
```

The chatbot calls llama.cpp over a private Docker network. The llama.cpp API port is not exposed to the faculty LAN. Only the Streamlit port is published.

NOTE

The existing Python code still uses variable names such as `LMSTUDIO_BASE_URL`. That name is retained only for compatibility; the URL now points to the llama.cpp Vulkan container, not LM Studio.

No monitor is required on the server. Vulkan compute via the RADV driver works through the render device node (`/dev/dri/renderD*`) without a display attached — this works identically on a headless Ubuntu Server install.

2. What You Should Bring / Use

- The populated deployment folder containing `data/chroma_db/chroma.sqlite3` and all UUID folders.
- The updated shared-server deployment kit for this Vulkan architecture (docker-compose.yml must reference the AMD services below, not the old NVIDIA ones).
- The latest `requirements.txt` based on the working `chatbotui` environment (unchanged — this runs on CPU regardless of GPU vendor).
- A few known-good thesis questions for final testing.

⚠WARNING

Do not use any deployment kit whose `docker-compose.yml` references `runtime: nvidia`, `nvidia/cuda`, or `network_mode: host`.

PHASE A — Read-Only Safety Check

These commands do not install, remove, or restart anything. Paste the whole block:

```
echo "==== USER ===="
whoami
id
echo "==== DOCKER ===="
docker --version
docker compose version
echo "==== EXISTING CONTAINERS ===="
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
echo "==== GPU DEVICE NODES ===="
ls -l /dev/dri /dev/kfd 2>&1
echo "==== GROUPS ===="
getent group render video
echo "==== KERNEL ===="
uname -r
echo "==== PORT 18501 CHECK ===="
ss -ltn | grep ":18501 " || true
```

✓CHECKPOINT A

- Docker version and Compose version should display normally.
- Existing faculty containers should remain running.
- `/dev/dri` and `/dev/kfd` should both exist and list device files.
- Your user should ideally already belong to the `render` and/or `video` groups. If not, this is a one-line host change (`usermod -aG render,video $USER`) — ask faculty IT to make it rather than doing it yourself on a shared machine.
- If the final port command prints nothing, port 18501 is probably free.

⛔STOP

If `/dev/dri` or `/dev/kfd` don't exist, or Docker commands fail, STOP and ask faculty IT whether the `amdgpu` driver is loaded. Do not install ROCm or reinstall Docker yourself.

PHASE B — Test GPU Access in Docker Without Changing the Host

This runs a temporary container only. It does not install anything on the host — it installs Vulkan tooling *inside* a throwaway container to check that the GPU is visible.

```
docker run --rm \
  --device=/dev/dri \
  --device=/dev/kfd \
  --group-add video \
  --group-add render \
  ubuntu:24.04 bash -c "apt-get update -qq && apt-get install -y -qq mesa-vulkan-drivers vulkan-tools >/dev/null && vulkaninfo --
summary"
```

✓CHECKPOINT B

- Output should list a device similar to `Radeon 8060S Graphics (RADV GFX1151)` .

⛔STOP

If you see `llvmpipe` only (software rendering, no real GPU), `No devices found` , or a permissions error on `/dev/kfd / /dev/dri` , STOP and ask faculty IT to confirm the `amdgpu` kernel driver and your group membership. gfx1151 is recent silicon — if the GPU isn't detected at all, the most common cause is a kernel that predates full gfx1151 support; ask IT about installing Ubuntu's HWE (Hardware Enablement) kernel, which is a supported, low-risk update path rather than a custom kernel build. Do not modify the Docker daemon yourself.

PHASE C — Prepare the Chatbot Folder

Use Ubuntu Files (GUI) or `scp / rsync` (headless server) to copy the complete deployment folder into your home folder, e.g.:

```
/home/YOUR_USERNAME/DBP_Chatbot_Deploy_SharedServer
```

⚠WARNING

`YOUR_USERNAME` is an example placeholder. Do not type it literally.

```
cd ~/DBP_Chatbot_Deploy_SharedServer
pwd
ls -la
ls -lh data/chroma_db/chroma.sqlite3
```

✓CHECKPOINT C

- `pwd` should end with `DBP_Chatbot_Deploy_SharedServer` .
- `docker-compose.yml` , `Dockerfile` , `requirements.txt` , and `.env` should be visible.
- `data/chroma_db/chroma.sqlite3` must exist directly at that path.

⛔STOP

If `chroma.sqlite3` is under `data/chroma_db/chroma_db/` , fix the extra folder level before continuing.

PHASE D — Review the New Configuration Before Running Anything

```
cat .env
echo
cat docker-compose.yml
```

The important `.env` values should include:

```
LMSTUDIO_BASE_URL=http://llm:8080/v1/
LMSTUDIO_MODEL=google/gemma-3-4b
CHATBOT_HOST_PORT=18501
LLAMA_HF_REPO=lmstudio-community/gemma-3-4b-it-GGUF:Q4_K_M
```

The GGUF repo line is **identical to the NVIDIA edition** — the model file itself doesn't change, only how it's run.

The `docker-compose.yml` should look like this for the `llm` service (abbreviated):

```

services:
  llm:
    image: ghcr.io/ggml-org/llama.cpp:server-vulkan
    devices:
      - /dev/dri
      - /dev/kfd
    group_add:
      - video
      - render
    volumes:
      - llama_models:/root/.cache/llama.cpp
    command: >
      -hf ${LLAMA_HF_REPO}
      --host 0.0.0.0
      --port 8080
      -ngl 99
    networks:
      - dbp-internal
    restart: unless-stopped

  chatbot:
    build: .
    ports:
      - "${CHATBOT_HOST_PORT}:8501"
    env_file: .env
    volumes:
      - ./data/chroma_db:/app/data/chroma_db:ro
    networks:
      - dbp-internal
    depends_on:
      - llm
    restart: unless-stopped

networks:
  dbp-internal:
    driver: bridge

volumes:
  llama_models:

```

✓ CHECKPOINT D

- The Compose file should contain two services named `llm` and `chatbot` .
- It should **NOT** contain `network_mode: host` , `runtime: nvidia` , or any `nvidia.com/gpu` device reservation.
- The `llm` service **should** contain the `devices:` and `group_add:` lines shown above.

PHASE E — Pull the llama.cpp Vulkan Image

This does not touch existing faculty containers. It only downloads the image required by this Compose project.

```
docker compose -p dbp-chatbot pull llm
```

✓ CHECKPOINT E

- The command should finish without an ERROR. The Vulkan server image is small (roughly 1-2 GB) compared to a CUDA or ROCm image.

⚠ STOP

If registry access is blocked or the pull fails, copy the exact error. Do not change Docker settings.

PHASE F — Build Only the Chatbot Container

This builds your Python/Streamlit/RAG image from the Dockerfile. This part is unchanged from the NVIDIA edition — BGE-M3 and the reranker still run on CPU.

```
docker compose -p dbp-chatbot build chatbot
```

✓CHECKPOINT F

- The build must finish successfully.

🛑STOP

If it fails, run the same build again only after identifying the error. Do not run apt/pip installs directly on Ubuntu.

PHASE G — Start ONLY the LLM Container First

```
docker compose -p dbp-chatbot up -d llm
docker compose -p dbp-chatbot ps
docker compose -p dbp-chatbot logs -f --tail=100 llm
```

The first start may download the Gemma GGUF into this project's Docker volume. Wait for the model to finish downloading/loading.

NOTE

To exit the live log view without stopping the container, press Ctrl+C.

✓CHECKPOINT G

- The `llm` service should become healthy/running.
- Logs should show something like `ggml_vulkan: Found 1 Vulkan devices` and `Radeon 8060S Graphics (RADV GFX1151)` — **not** `llvmpipe`.

🛑STOP

If logs show `ggml_vulkan: No devices found`, the model repeatedly restarts, or only `llvmpipe` appears (meaning it's running on the CPU's software renderer, not the real GPU), STOP and capture the logs. Common causes: Phase B wasn't actually passed, the `devices: / group_add:` lines are missing from `docker-compose.yml`, or the host user isn't in the `render / video` groups. Do not restart the host Docker daemon.

PHASE H — Test llama.cpp from Inside the Docker Network

Port 8080 is intentionally not published to the host, so test it from a temporary container attached to the same Compose network:

```
docker run --rm --network dbp-chatbot_dbp-internal curlimages/curl:latest -fsS http://llm:8080/v1/models
```

You should receive JSON describing the loaded model.

NOTE

If the network name differs, run `docker network ls | grep dbp` and use the exact name shown.

PHASE I — Verify ChromaDB

```
docker compose -p dbp-chatbot run --rm --no-deps chatbot python /app/verify_chroma.py
```

Expected result (unchanged — this step has nothing to do with the GPU backend):

```
Collection      : dbp_khidmatnasihat_clean_atomic
Document count  : 33320
Expected count  : 33320
```

🛑STOP

If the count is not exactly 33,320, STOP. Do not rebuild or re-embed the knowledge base.

PHASE J — Start the Full Chatbot Project

```
docker compose -p dbp-chatbot up -d
docker compose -p dbp-chatbot ps
docker compose -p dbp-chatbot logs --tail=150 chatbot
```

✓CHECKPOINT J

- `llm` should be running/healthy.
- `chatbot` should be running/healthy after startup.
- Existing faculty containers must remain unchanged.

PHASE K — Open the Chatbot

If you’re on the GUI machine directly

Open a browser on that machine:

```
http://localhost:18501
```

If you’re on the headless server (no monitor/desktop)

Use an SSH tunnel from your own laptop rather than exposing the port on the LAN yet — this mirrors the “don’t expose publicly on day one” caution from Phase N:

```
ssh -L 18501:localhost:18501 your_username@faculty-server-ip
```

Then open `http://localhost:18501` in the browser **on your own laptop** (not the server).

Only move to direct LAN access (`http://<server-LAN-IP>:18501`) once faculty IT has confirmed firewall rules for this — see Phase N.

Test known thesis questions, including one normal advice question and one refusal/out-of-domain question.

NOTE

For now, use port 18501 rather than 8501 to reduce the chance of clashing with an existing application.

PHASE L — Useful Commands for a Beginner

What you want to do	Command
See only our containers	<code>docker compose -p dbp-chatbot ps</code>
See chatbot logs	<code>docker compose -p dbp-chatbot logs --tail=200 chatbot</code>
See LLM logs	<code>docker compose -p dbp-chatbot logs --tail=200 llm</code>
Follow logs live	<code>docker compose -p dbp-chatbot logs -f</code>
Check GPU is actually being used	Look for “Vulkan devices found... RADV GFX1151” in <code>llm</code> logs (see Phase G)
See GPU utilization (if IT allows installing it)	<code>radeontop</code> — an open-source AMD GPU monitor, available via <code>apt install radeontop</code> ; this is a monitoring tool only, not a driver change
See our resource use	<code>docker stats --no-stream</code>
Restart only our chatbot	<code>docker compose -p dbp-chatbot restart chatbot</code>
Stop/remove only our running project	<code>docker compose -p dbp-chatbot down</code>
Start our project again	<code>docker compose -p dbp-chatbot up -d</code>

STOP

Do NOT run `docker system prune` , `docker container prune` , `docker volume prune` , or commands that stop/remove all containers. This server belongs to other applications too.

WARNING

When stopping your project, do not add `-v` to `docker compose down` unless you intentionally want to delete the Gemma/model cache volume.

PHASE M — If Port 18501 Is Already Used

```
ss -ltn | grep ":18501 " || true
```

If something is using it, choose another unused high port, e.g. 18502 or 18510. Edit only this line in `.env`:

```
CHATBOT_HOST_PORT=18510
```

Then:

```
docker compose -p dbp-chatbot up -d
```

PHASE N — External Faculty URL Comes Later

Do not expose the chatbot publicly during the first local installation. First confirm the SSH-tunnel/localhost test works and the known thesis answers behave correctly.

- Faculty IT can later map a fixed IP/domain to the Streamlit service.
- Use HTTPS and access control for external data-provider testing.
- Do not publish the llama.cpp Vulkan port 8080 to the Internet.
- Do not modify the faculty firewall/reverse proxy without their administrator.

Troubleshooting Decision Table

Symptom	Meaning	Action
<code>/dev/dri</code> or <code>/dev/kfd</code> missing (Phase A)	<code>amdgpu</code> driver not active / very old kernel	Ask faculty IT; do not install ROCm or a custom kernel yourself
Phase B shows <code>llvmpipe</code> only	GPU not reachable from Docker, falling back to software rendering	Check device passthrough and group membership; ask IT to confirm the <code>amdgpu</code> driver is loaded
<code>ggml_vulkan: No devices found</code> in <code>llm</code> logs	Device nodes not passed into the container correctly	Re-check <code>devices:</code> / <code>group_add:</code> in <code>docker-compose.yml</code> match Phase D; some hosts need <code>/dev/dri/renderD128</code> listed individually instead of the whole <code>/dev/dri</code> directory — ask IT if this persists
<code>llm</code> image pull fails	Registry/network restriction	Capture error; do not modify global Docker config
<code>llm</code> loads model but stays unhealthy	Startup issue unrelated to GPU	Check <code>docker compose -p dbp-chatbot logs -tail=200 llm</code>
Any ROCm/HIP-related error appears	Wrong image pulled	This guide uses <code>server-vulkan</code> , not <code>server-rocm</code> — verify the image tag in <code>docker-compose.yml</code>
Chroma count is not 33,320	Wrong data folder/mount	Fix folder path; do not re-index
Chatbot cannot call LLM	Internal Compose networking/model health issue	Check both service logs and <code>/v1/models</code> from the <code>dbp-internal</code> network (Phase H)
<code>localhost:18501</code> (or SSH tunnel) works	Core deployment succeeded	Only then discuss LAN/domain access (Phase N)
Other faculty apps misbehave	Potential shared-host impact	Stop only our project with <code>docker compose -p dbp-chatbot down</code> and inform IT

Emergency: Remove Only Our Running Containers

From inside the DBP deployment folder:

```
docker compose -p dbp-chatbot down
```

This stops/removes the containers and network belonging to this Compose project. It does not intentionally stop the faculty’s unrelated containers.

 **WARNING**

Do not run global cleanup commands on a shared Docker server.

Official Technical Basis

Docker Compose is intended for defining and running multi-container applications, and Docker supports passing through host devices (such as `/dev/dri` and `/dev/kfd`) to containers without a vendor-specific container runtime. llama.cpp provides an OpenAI-compatible HTTP server with official pre-built Docker images for multiple backends, including a Vulkan variant (`ghcr.io/ggml-org/llama.cpp:server-vulkan`) that uses the open-source Mesa RADV driver — this is currently the best-performing backend on AMD's Strix Halo (gfx1151) integrated GPU, ahead of the ROCm/HIP backend for this specific chip.

References: Docker device passthrough docs; llama.cpp server/Docker docs (`docs/docker.md`); Mesa RADV driver documentation; AMD Ryzen AI Max+ 395 / Strix Halo (gfx1151) community benchmarks.

OK: Today's goal: reach a working local chatbot (via `localhost:18501` on the GUI machine, or an SSH tunnel on the headless server) without installing or changing host-level AI/GPU software, and without disturbing the faculty's existing Docker applications.